future internet

MDPI

Article

Elasticsearch-Based Threat Hunting to Detect Privilege Escalation Using Registry Modification and Process Injection Attacks

Akashdeep Bhardwaj [1,*], Luxmi Sapra [2] and Shawon Rahman [3,*]

[1] Centre for Cybersecurity, School of Computer Science, University of Petroleum and Energy Studies, Dehradun 248007, India
[2] Faculty Computer Application, Graphic Era Hill University, Dehradun 248002, India; luxmi.sapra@gmail.com
[3] Department of Computer Science, University of Hawaii-Hilo, Hilo, HI 96720, USA
* Correspondence: abhardwaj@ddn.upes.ac.in (A.B.); srahman@hawaii.edu (S.R.)

Abstract

Malicious actors often exploit persistence mechanisms, such as unauthorized modifications to Windows startup directories or registry keys, to achieve privilege escalation and maintain access on compromised systems. While information technology (IT) teams legitimately use these AutoStart Extension Points (ASEPs), adversaries frequently deploy malicious binaries with non-standard naming conventions or execute files from transient directories (e.g., Temp or Public folders). This study proposes a threat-hunting framework using a custom Elasticsearch Security Information and Event Management (SIEM) system to detect such persistence tactics. Two hypothesis-driven investigations were conducted: the first focused on identifying unauthorized ASEP registry key modifications during user logon events, while the second targeted malicious Dynamic Link Library (DLL) injections within temporary directories. By correlating Sysmon event logs (e.g., registry key creation/modification and process creation events), the researchers identified attack chains involving sequential registry edits and malicious file executions. Analysis confirmed that Sysmon Event ID 12 (registry object creation) and Event ID 7 (DLL loading) provided critical forensic evidence for detecting these tactics. The findings underscore the efficacy of real-time event correlation in SIEM systems in disrupting adversarial workflows, enabling rapid mitigation through the removal of malicious entries. This approach advances proactive defense strategies against privilege escalation and persistence, emphasizing the need for granular monitoring of registry and filesystem activities in enterprise environments.

Keywords: privilege escalation; persistence; windows registry; startup directory; AutoStart Extension Point (ASEP); Sysmon; event correlation; SIEM; elasticsearch; threat hunting; malicious DLL; boot logon autorun; registry key modification; forensic analysis; attack detection

check for updates

Academic Editors: Pengfei Qiu and Eirini Eleni Tsiropoulou

Received: 8 July 2025
Revised: 12 August 2025
Accepted: 23 August 2025
Published: 29 August 2025

Citation: Bhardwaj, A.; Sapra, L.; Rahman, S. Elasticsearch-Based Threat Hunting to Detect Privilege Escalation Using Registry Modification and Process Injection Attacks. *Future Internet* **2025**, *17*, 394. https://doi.org/10.3390/fi17090394

Correction Statement: This article has been republished with a minor change. The change does not affect the scientific content of the article and further details are available within the backmatter of the website version of this article.

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

The need for strong cybersecurity measures has increased significantly with the expanding digital world. A thorough understanding and categorization of vulnerabilities are crucial in the constantly changing field of cybersecurity, where a single weakness can be disastrous for the entire network. With the growing dangers and complexity of modern infrastructure, there is a need for a systematic and standardized method to detect cyberattacks related to registry modifications that enable persistence and Privilege Escalation. Privilege escalation [1]

refers to exploiting system vulnerabilities to gain higher access to protected resources. First, attackers gain access to and move through an infrastructure using low-level permissions; however, they need higher permissions to carry out malicious actions that would otherwise be blocked. The main goals are to install malware or backdoors with elevated permissions to ensure long-term access to the system. One method used in this process is registry modification, which involves changing the Windows Registry to manipulate system behavior or gain unauthorized access to higher privilege levels.

The tactics related to registry modification for privilege escalation are complex. Attackers take advantage of misconfigurations or vulnerabilities in the registry to create new keys or change existing ones, which alters how system processes or applications behave. For example, by changing permissions on specific registry keys, an attacker can give themselves or others higher privileges. Additionally, registry keys are controlled to run malicious code during system startup or when certain applications open, enabling the attacker to execute arbitrary code with elevated privileges. This approach is particularly sneaky and can be carried out without extra malware, relying instead on the trust the operating system places in its registry.

Understanding the implications of registry modification tactics in privilege escalation requires a comprehensive analysis of the security measures in place within an operating system. Windows, for example, employs various security mechanisms, such as User Account Control (UAC) [2] and access control lists (ACLs) [3], to mitigate the risks associated with unauthorized registry modifications. Cybersecurity professionals must stay vigilant and proactive in monitoring registry changes, enforcing strict access controls, and educating users about the potential risks tied to privilege escalation tactics that exploit registry modifications.

The paper highlights the complexities of intrusion detection systems by showing how attackers exploit legitimate system functions, such as Windows registry keys and Dynamic Link Library (DLL) loading mechanisms, to gain privilege escalation and maintain persistence. Traditional detection methods often depend on static signatures or rule-based triggers, which can overlook evolving attack patterns that blend with normal operations. The Elasticsearch-based SIEM framework presented addresses these issues by correlating data from multiple sources, especially Sysmon Event IDs and Windows native logs, to reconstruct attack sequences in real time. Through detailed case studies, the research demonstrates how changes to registry keys and DLL drops in temporary directories can act as reliable indicators of compromise, particularly when combined with process lineage analysis. By validating detection accuracy with confusion matrix metrics and anomaly scores using the Z-score method, the work points out the strengths and limitations of current intrusion detection strategies. It emphasizes that effective systems must combine behavioral baselining with event correlation to detect stealthy threats. Additionally, the methodology shows that reducing false positives involves carefully filtering out benign activities while still capturing subtle malicious behaviors. Ultimately, this study advocates for adaptive, data-driven intrusion detection systems that evolve alongside adversarial tactics, enabling quicker response, more accurate threat identification, and targeted remediation within enterprise environments.

To illustrate the implications of privilege escalation through registry manipulation, three real-world examples are examined, each highlighting different methodologies and outcomes associated with this form of attack. The first example involves the notorious malware known as 'Stuxnet' [4], which was designed to target industrial control systems. Stuxnet utilized a sophisticated approach to privilege escalation by manipulating the Windows registry to inject malicious code into legitimate processes. By altering registry keys, the malware was able to execute its payload with higher privileges than those initially granted to it. This allowed Stuxnet to not only infiltrate the targeted systems but also

to manipulate the operations of critical infrastructure, demonstrating the potential for significant real-world consequences when privilege escalation is successfully executed through registry exploitation.

Another relevant case is the 'CVE-2016-0189' [5] vulnerability, which affected Microsoft Internet Explorer. This vulnerability allowed attackers to exploit the registry and escalate privileges, enabling them to perform process injection attacks. By creating a malicious web page that exploited this vulnerability, attackers could run arbitrary code within the user's session. The exploitation of this vulnerability highlighted the importance of keeping software updated and the risks associated with unpatched systems. The ability to manipulate the registry to gain higher privileges demonstrates the complex link between software vulnerabilities and privilege escalation tactics used by cybercriminals. The 'EternalBlue' exploit, which was part of the NSA's leaked hacking tools, is a notable example of privilege escalation through registry manipulation. This exploit targeted a weakness in the Windows Server Message Block (SMB) protocol, allowing attackers to run remote code and then manipulate the registry to inject malicious processes. The widespread impact of EternalBlue was felt worldwide, as it helped spread ransomware rapidly, especially the 'WannaCry' [6] attack. This event underscored the urgent need for strong security measures and the serious consequences when malicious actors effectively use privilege escalation techniques. Each of these examples shows the complex nature of privilege escalation and underscores the importance of understanding and reducing these risks in today's cybersecurity practices.

Process injection [7] enables an attacker to run code within the address space of another process. This technique is especially effective in Windows environments, where many applications operate with elevated privileges. By injecting code into a trusted process, attackers can bypass security measures and execute malicious payloads without suspicion. When combined with Registry manipulation, process injection becomes a powerful tool for privilege escalation. For example, attackers modify a Registry key to ensure a malicious executable runs during system startup or when a specific application is launched. This approach not only facilitates initial compromise but also ensures persistence, allowing the attacker to maintain access even after system reboots.

The consequences of privilege escalation through Registry manipulation and process injections are severe, as they can result in complete system compromise. Once an attacker gains elevated privileges, they can access sensitive data, install additional malware, or create backdoors for future access. Organizations must adopt strong security measures to counter these threats, such as regularly monitoring Registry changes, implementing application whitelisting, and using advanced endpoint protection solutions. Additionally, educating users about the risks of running untrusted software and keeping systems up to date greatly lowers the chances of successful privilege escalation attacks. Understanding the details of these techniques is crucial for cybersecurity professionals defending against increasingly sophisticated threats.

When malware remains inactive on a host, it must be triggered to execute at a future time; this is known as Persistence [8]. Typical examples of persistence mechanisms include AutoStart keys [9] in the Registry, Scheduled Tasks [10], or Cron jobs, as well as Boot-time redirection. Malware attacks use such persistence tactics to activate before performing actions that could alert security professionals to begin their investigations. To do this, adversaries exploit vulnerabilities or misconfigurations to obtain administrator or root privileges. With elevated access, adversaries install malicious services or modify startup scripts to run payloads each time the system starts. For instance, Rootkits installed in the kernel ensure that the adversary's commands and presence remain hidden from detection tools. The abstract code for creating a backdoor to escalate privileges is provided in Table 1 for reference.

Table 1. Listing for Installing Backdoor.

```
def install_backdoor():
# Escalate privileges
if escalate_privileges():
# Write a malicious script to startup
with open("/etc/rc.local", "a") as startup_script:
startup_script.write("nohup/usr/bin/backdoor &")
print("Backdoor installed.")
else:
print("Privilege escalation failed.")
```

Another malicious goal is to access and extract sensitive data, such as confidential files, passwords, or cryptographic keys [11]. Elevated permissions give attackers access to restricted directories and files, enabling them to steal sensitive information like configuration files or encrypted password databases. For instance, dumping the contents of the '/etc/shadow' folder from a Linux system to crack passwords, the general method for exfiltrating such data is shown in Table 2 for reference.

Table 2. Listing Sensitive Data Exfiltration.

```
def exfiltrate_sensitive_data():
if escalate_privileges():
shadow_file = "/etc/shadow"
with open(shadow_file, "r") as file:
data = file.read()
send_to_adversary(data)
else:
print("Insufficient privileges.")
```

To demonstrate the real-world impact of privilege escalation through process injection, two notable examples will be examined, showcasing the methods used and the outcomes of these attacks. The first example features the notorious 'Mimikatz' [12] tool, which has been widely employed in various cyberattacks to extract plaintext passwords, Kerberos tickets, and other sensitive information from Windows systems. Mimikatz uses process injection techniques to manipulate the Local Security Authority Subsystem Service (LSASS), which manages security policies on the system. By injecting code into LSASS, attackers access its memory to retrieve authentication credentials without raising alarms. This approach has been especially effective in enterprise environments, where attackers use stolen credentials to move laterally across the network, escalate privileges, and access critical systems and data.

The second major case is the 'Cobalt Strike' [13] framework, which is used by penetration testers but has also been adopted by cybercriminals for malicious purposes. Cobalt Strike uses process injections to run payloads within legitimate processes, helping it avoid detection by security software. One of its techniques, known as 'Beacon', lets attackers establish a command-and-control channel by injecting code into processes like PowerShell or svchost.exe. This feature not only allows remote access but also helps attackers escalate privileges by running commands with the same permissions as the injected process. Cobalt Strike's ability to use process injection has made it a popular tool among threat actors, resulting in numerous high-profile breaches and data exfiltration events.

To address these risks, the MITRE framework [14] has developed a platform for threat detection that explains and categorizes adversarial tactics and relevant defensive techniques; however, it is known to fall short of providing the necessary mitigation tactics. This research presents a comprehensive investigation into threats related to Persistence and Privilege

Escalation. It utilizes a dataset from an organization to improve our understanding and application of these critical tools. The highlights of this research are:

- Design an Elasticsearch SIEM for threat hunting to identify privilege escalation and persistence.
- Detect modifications to the Boot Logon Autorun (ASEP) registry key that occur during user login.
- Detect malicious DLLs in the Temp directory through Sysmon Event Code analysis.
- Emphasize correlating events to identify and prevent attacks by recognizing malicious registry entries.

2. Literature Review

Branescu et al. (2024) [15] conducted CVE mapping to the applicable 14 MITRE ATT&CK tactics using advanced transformer-based models. Different encoder architectures, such as generative large-scale models, address this multilabel classification. The models performed well, with the best score achieved using the SecRoBERTa model, confirmed by an F1 score of 77.81%, closely followed by SecBERT (78.77%), CyBERT (78.54%), and TARS (78.01%). In contrast, GPT-4's performance in zero-shot setups was relatively lower, with an F1 score of 22.04%. The authors conducted a thorough error analysis to better understand the models' performance and weaknesses. The code used in all experimental configurations was made available as open-source.

Kanan et al. [16] conducted a thorough review of cyberattack datasets, finding that no single dataset fully serves the purpose for any specific task. The research highlighted the importance of careful selection, especially in cyber studies, as research questions need to closely align with their respective datasets. Recognizing ongoing efforts to foster cooperation and innovation among communities involved in creating datasets that reflect the ephemeral nature of cyber threats, the authors pointed out that critical analysis of dataset properties and issues is a key approach that academia, industry, and dataset producers will use as steps toward developing a better cybersecurity framework and system that can more resiliently handle a highly dynamic and ambiguous landscape.

An intelligent cybersecurity pen test assistant was developed by Pratama et al. [17] for ethical researchers. It is a large language model specifically trained to support penetration testing tasks as a chatbot. It benefits from excellent articles about vulnerable machines, hacking techniques, and documentation of open-source penetration testing tools, all enhanced with an expert response structure. In addition to providing a realistic and rigorous standard for assessing LLMs' technical knowledge, reasoning skills, and practical usefulness in dynamic penetration testing scenarios, our research fills a significant gap in traditional cybersecurity Q&A standards.

Tchimwa et al. [18] proposed a tripartite ranking algorithm to evaluate three main components of a logical attack graph: vulnerabilities, privileges, and potential attack exploits. Since each node type was unique with specific attributes and impacts on the system's security, the authors ranked them collectively, taking into account the interdependence between nodes in the attack graph. Using the proposed ranking scheme, they could assign a numerical value to each node based on its type; this serves as a clear indicator of its importance to an attacker.

Ryu et al. [19] examined the cybersecurity risks in energy IT infrastructure related to digital transformation and network growth. As these advancements boost efficiency, energy networks become more susceptible to cyber threats, including malware attacks like Industroyer, Triton, NotPetya, and BlackEnergy. This paper emphasizes the main vulnerabilities in energy systems and discusses some of these malware threats. The study underscores the need for stronger system integrity checks, network segmentation, and

anomaly detection. It also presents a multi-layered security architecture, where each layer provides separate protection and works together toward a comprehensive defense. Overall, the study highlights the importance of integrating organizational, technical, and policy-based solutions to improve cybersecurity across all levels of energy infrastructure.

He et al. [20] proposed a multidimensional detection framework for identifying lateral movement behavior within intranet environments, based on the SMB protocol. The framework detects attack samples used by adversaries in their lateral movement through neural networks, active trapping, and passive scanning. In a simulated environment, they assessed the effectiveness of the active trapping technique and verified with real malware samples that the neural network detection accuracy was nearly 90%. The experimental results, based on the SMB protocol, showed that the proposed framework can effectively detect lateral movement activity within an intranet.

The transformative impact of LLMs on addressing major cybersecurity issues was studied by Karsi et al. [21]. Traditional security methods often fail to detect, contain, and respond to complex threats because of the rapid evolution of digital environments and the growing sophistication of cyber threats. Thanks to their exceptional natural language processing abilities, LLMs can manage large datasets, spot vulnerabilities, and automate threat detection. Their applications go beyond creating security policies, malware analysis, phishing detection, and incident response. By utilizing advanced features like context awareness and real-time adaptation, LLMs strengthen an organization's defenses against cyberattacks and support better decision-making.

To analyze network forensics, Paracha et al. [22] utilized the capabilities of modern AI systems based on ML and DL algorithms. Using AI/ML approaches, the authors proposed a model for an investigation that examines traffic and behavioral patterns to identify cyberattacks as either past or potential. The speed of investigations has increased thanks to an AI-based network forensics model; as a result, network monitoring has become more efficient without the need for manual effort. This also aims to provide network managers with quick and reliable information to make prompt and effective decisions, helping to prevent and mitigate future intrusions.

Smiliotopoulos et al. [23] provided a detailed and thorough description of how to use an intrusion detection system (IDS) to detect lateral movement, addressing emerging communication paradigms like the Internet of Things (IoT). The survey included 53 items collected over eight years, focusing on three main areas: graph-based tactics, machine learning solutions, and Endpoint Detection and Response (EDR) methods. The authors offered important insights that could assist in the study of lateral movement, along with interrelations and a timeline of progress in this field.

A hardware honeypot technique was introduced by Omar et al. [24] as an additional layer of protection against hardware Trojans (HTs). The proposed solution was implemented on a Raspberry Pi and tested using an FPGA-emulated HT circuit. To detect and mitigate HTs in IoT devices, this method employs hardware honeypots. The results show that the approach can identify and reduce HTs without adding complexity to IoT devices. With a flexible yet strong layer of security, full customization of the Trojan-agnostic solution ensured that security needs were met. This work enhanced the robustness of IoT networks and provided valuable insights into improving security in IoT devices against hardware-based cyberattacks. The increasing security concerns in IoT environments are expected to be addressed by this invention.

The rapid growth of information and communications technology has enabled digitally controlled, software-driven distributed energy resources (DERs) to improve grid flexibility, efficiency, and operational support. However, this progress has also increased exposure to cyber risks, including hardware and software flaws, communication vulnerabilities,

and human errors. Strengthening the cyber-resiliency of DER-based smart grids—their ability to withstand and recover from cyber intrusions—has thus become a key focus for both industry and academia. In a recent survey, Liu et al. [25] reviewed advances in cyber-resiliency enhancement (CRE) for DER-based smart grids, presenting an integrated framework and outlining future research directions. Their work includes tailored threat modeling for hierarchical DER systems, in-depth examination of vulnerabilities, and analysis of defense-in-depth strategies spanning prevention, detection, mitigation, and recovery. They also identify five key resiliency enablers within a comprehensive CRE framework and highlight urgent challenges, aiming to guide ongoing innovation in protecting the next generation of smart grid infrastructure.

Zhang et al. [26] presented the first in-depth review dedicated to the security of Machine Learning-based smart grid applications (MLsgAPPs), with a clear focus on their unique characteristics within power systems. The study begins by outlining how adversarial attacks are constructed for MLsgAPPs, followed by a dual-perspective vulnerability analysis—examining weaknesses from both the power system and ML model viewpoints. It then offers a structured comparison of existing research on adversarial attacks across different stages of the power grid, including generation, transmission, distribution, and consumption, alongside the corresponding defense strategies tailored to each threat type. The review also extends its analysis to the emerging risks posed by large language model-powered smart grid applications, such as those leveraging ChatGPT. Concluding with future research directions for both attackers and defenders, the work aims to inspire deeper exploration into securing MLsgAPPs against evolving adversarial threats.

Alsharabi et al. [27] examined how attackers perform lateral movement within a compromised network, especially focusing on methods that target credential access. Their work used a custom SIEM platform built on Elasticsearch, enhanced with Kibana Query Language (KQL) and Lucene queries to investigate these threats. With a realistic dataset, they simulated attacker tactics, techniques, and procedures to mimic real-world scenarios. This allowed the team to identify indicators of compromise and develop precise queries to detect suspicious lateral activity. The study provides practical insights into detection strategies, showing how a well-configured SIEM with advanced search capabilities can effectively identify and counter these threats.

Tracking suspicious activity and collecting digital evidence manually often takes a lot of time, can be inconsistent, and depends heavily on the investigator's expertise. System logs are an essential source of such evidence, but in Microsoft Windows, they can have irregular event flow and are hard to audit. To address this, Kim et al. [28] introduced a model designed to analyze Windows logs efficiently. Their approach extracted key and shared event lists, which enabled detailed activity tracking. Demonstrated through illegal file access detection, the method used Elastic Stack with a three-step template for event analysis and visualization, giving analysts the ability to adjust investigation depth for more flexibility and accuracy.

3. Research Methodology

The research methodology in this study focuses on developing and implementing an Elasticsearch-based [29] Security Information and Event Management (SIEM) system. The authors created this for threat hunting to detect registry modifications that enable privilege escalation on compromised machines. The first phase of the threat hunting process centered on identifying Boot Logon Autorun mechanisms, especially targeting the AutoStart Extension Point (ASEP) registry keys [30]. These keys are important because they control the execution of specific processes when a user logs in, making them potential targets for malicious activities. By carefully analyzing these registry entries, the research

aimed to find unauthorized changes that could signal an ongoing attack or a previously compromised system.

The dataset for this research was assembled from a controlled, enterprise-like environment within an institutional network. Log data was collected from 25 Windows 10 Pro endpoints and 3 Windows Server 2019 instances, set up to simulate both typical user workstations and administrative servers. These systems were joined to a local Active Directory domain to mimic realistic authentication and policy enforcement scenarios. Host-level telemetry was collected using Sysmon v13 (with a custom XML configuration for detailed registry, file, and process monitoring) along with native Windows Event Logging. Logs were transmitted via Winlogbeat 8.x agents over encrypted channels to a central Ubuntu 22.04 LTS Elasticsearch node (8 vCPU, 32 GB RAM, 2 TB SSD). Kibana was hosted on the same node for querying and visualization, while Lucene syntax was used for advanced filtering and correlation. The dataset included 925,737 log entries over a 30-day observation period, with both simulated malicious activities (such as DLL drops and registry modifications) and normal baseline activity. This dual approach ensured that the detection logic could be tested against both attack scenarios and typical operational patterns.

Figure 1 shows the detailed workflow diagram for the entire methodology of the proposed Elasticsearch-based SIEM threat hunting framework. It starts with log sources, including Sysmon event IDs and Windows Event Logs, collected from all monitored endpoints. These logs are processed through the data ingestion layer (Filebeat/Winlogbeat) into Elasticsearch, then normalized and enriched to organize fields and connect related process or registry data. The workflow then splits into two hypothesis-driven hunts—dll drop detection in AppData Temp directories and autorun registry key abuse detection. Finally, the event correlation engine combines hunt results to rebuild attack chains, producing actionable alerts for SOC triage and incident response.

The research shifted focus to identifying malicious Dynamic Link Libraries (DLLs) located in the Temp directory. This part of the study was essential because attackers often use the Temp directory to run payloads without immediate suspicion. The methodology involved a thorough analysis of Sysmon Event Codes, which offered a detailed record of system activities and changes. By correlating these events, the study was able to trace a series of actions that usually occur before unauthorized modifications to Windows registry keys. This correlation not only helped identify malicious activities but also highlighted the need for ongoing monitoring and analysis of system events to spot anomalies that may indicate a compromise.

This research approach highlights the importance of event correlation in improving the detection and response to security threats. By confirming successful changes to registry keys through Sysmon Events, Table 3 shows that quick identification and fixing of malicious entries greatly lessen the impact of attacks. The results stress the need for organizations to adopt strong threat hunting practices, using advanced tools like Elasticsearch SIEM to find and address potential security issues proactively. This proactive method not only helps detect threats early but also boosts the system's overall strength against future attacks, creating a safer computing environment.

This research presents the SIEM built on the Elasticsearch platform, running on an Ubuntu OS. The implementation used Python and query languages like Kibana [31] and Lucene [32], ingesting a total of 925,737 log entries collected over a 30-day period from 25 Windows 10 Pro workstations and 3 Windows Server 2019 instances within an institutional Active Directory domain. This SIEM dashboard features a query menu to modify field names, searches, and filters to perform pivots by submitting domain-specific languages like KQL or Lucene queries, as illustrated in the accompanying Figure 2.

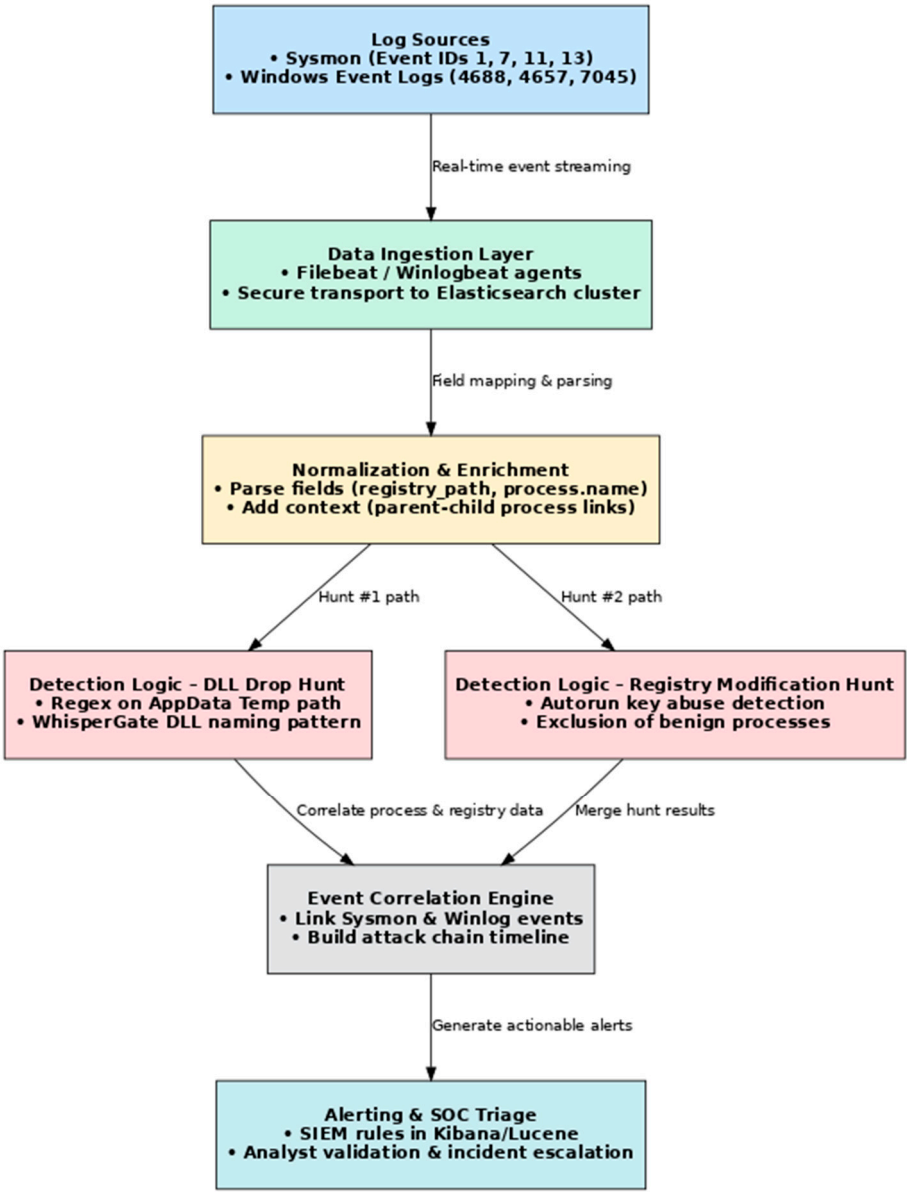


Figure 1. SIEM Methodology Workflow.

Table 3. Windows & Sysmon Event IDs.

Event Type	Event ID	Name/Description	Relevance to Research
Sysmon	1	Process Creation—Logs details when a process is created, including command line, parent process, and hashes.	Used to trace malicious process executions, such as PowerShell.exe and Rundll32.exe calls.
Sysmon	7	Image Loaded—Records DLLs loaded into processes.	Key for detecting malicious DLL injections in process memory.
Sysmon	10	Process Access—Captures when a process opens another process's memory for read/write.	Used to detect process injection attempts.
Sysmon	11	File Creation—Logs newly created files.	Helps identify DLLs or binaries dropped in suspicious directories (e.g., AppData\Temp).

Table 3. *Cont.*

Event Type	Event ID	Name/Description	Relevance to Research
Sysmon	12	Registry Object Created/Deleted—Logs creation or deletion of registry keys.	Detects creation of persistence-related registry entries.
Sysmon	13	Registry Value Set—Logs modifications to registry values.	Critical for detecting autorun key abuse and privilege escalation.
Windows Event Log	4688	Process Creation (Security Log)—Records creation of new processes with detailed command-line arguments.	Correlates with Sysmon ID 1 to verify process execution.
Windows Event Log	4657	Registry Value Modification—Logs when registry values are modified.	Supports persistence and privilege escalation detection logic.
Windows Event Log	4663	Object Access—Tracks file/folder access permissions and usage.	Provides context on suspicious file activity linked to malicious processes.
Windows Event Log	4689	Process Termination—Logs when a process exits.	Useful for reconstructing process chains and attack timelines.
Windows Event Log	7045	Service Creation—Records creation of Windows services.	Helps detect service-based persistence or privilege escalation mechanisms.

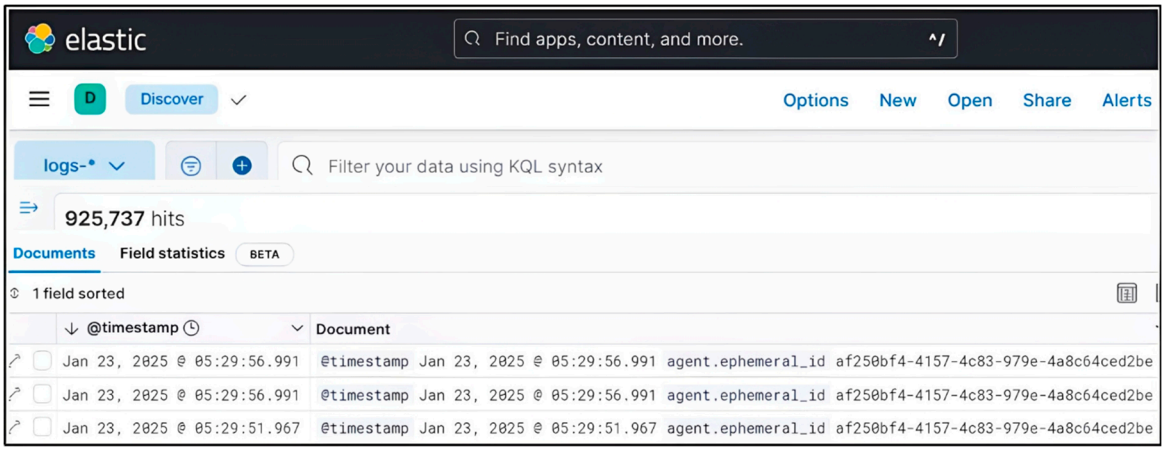


Figure 2. Elasticsearch SIEM Dashboard with logs.

The SIEM dashboard features an intuitive query menu that allows users to easily manipulate field names and run searches and filters. This feature is essential for performing data pivots, which enable analysts to examine logs in greater detail. Users submit queries using domain-specific languages, enhancing the system's flexibility and ability to handle complex search needs. The option to customize queries not only simplifies data retrieval but also helps users extract meaningful insights from the large volumes of log data available.

4. Research Performed

4.1. Threat Hunt #1 DLL File Drop in AppData Directory

The researchers found evidence that a dynamic link library (.dll) file was placed in the 'AppData\Local emp' directory, matching the WhisperGate naming pattern. This led to the discovery of a PowerShell encoded command, registry key modification, and suspicious network connections using a port associated with Trojans and other malware. The authors also discovered that persistence was maintained through the Windows Auto Run registry key, which overlapped with the initial investigation. Malware commonly performs download and execution from temporary or AppData locations, as these are paths where

user-level access is granted in Windows environments, making execution straightforward and predictable. A notable attack using this technique was observed in January 2022, involving the WhisperGate malware [33], targeting the Ukrainian government. The malware disguises itself as ransomware, but it is actually wiper malware that deletes files outside the user's home folder in the Windows directory.

The detection logic was validated through an atomic test, where a benign DLL file was intentionally copied into the \\AppData\\Local\\Temp directory and executed from there. This controlled action mimics the behavior of a DLL running from a temporary folder, a pattern commonly linked to malicious activity while ensuring the test remains safe for the environment. However, in some cases, this could be a legitimate file dropped during an application installation; nonetheless, such instances are mostly associated with malware or malicious executions. To address this, the authors created a query as shown in Table 4 to identify the DLL written to the AppData folder that matches the observed WhisperGate DLL naming.

Table 4. Query Logic for Threat Hunt #1.

Selection	Field	Value
DLL path (all)	File path	*AppData\Local\Temp*, *.dll*
DLL Regex	Naming schema	.*([Aa][Pp][Pp][Dd][Aa][Tt][Aa]\\[Ll][Oo][Cc][Aa][Ll]\\[Tt][Ee][Mm][Pp]\\)([a-zA-Z0-9]{8}.[Dd][Ll][Ll])

The authors provide the abstract code in Table 5 that searches for specific file paths in a dataset. It first creates a regular expression pattern, queryPattern, designed to match file paths pointing to DLL files in the AppData\Local\Temp directory. This pattern is case-insensitive and specifically targets filenames with eight alphanumeric characters followed by a .DLL extension. The function then loops through each file in the dataset, checking if either the file_path or file.path field matches the pattern. If a match occurs, the function immediately returns the corresponding file. This method is likely used to detect potentially malicious DLL files in temporary directories, which is common in malware analysis and cybersecurity investigations.

Table 5. Code to search for DLL files.

```
Begin
Define queryPattern as "/.*([Aa][Pp][Pp][Dd][Aa][Tt][Aa]\\\\[Ll][Oo][Cc][Aa][Ll]\\\\[Tt][Ee][Mm][Pp]\\\\)([a-zA-Z0-9]{8})\\.[Dd][Ll][Ll]/"
Define searchFields as ["file_path", "file.path"]
Function SearchFiles()
For each file in the dataset
If file.field("file_path") Matches queryPattern OR file.field("file.path") Matches queryPattern
Return file //Match found
Endif
End For
End Function
End
```

Using this logic, the author ran a KQL query parsing the logs to validate any DLL activities in the AppData directory as the path in focus, looking for target file names that include '.dll' as the extension. Figure 3 illustrates nine hits from the raw logs for any version of .DLL execution from within the system's AppData Temp directory.

@timestamp	Document
Jan 22, 2025 @ 23:14:46.589	file.path C:\Users\jamesmurphy\AppData\Local\Temp\kernel32.dll @timestamp Jan 22, 2025 @ 23:14:46.589 ag
Jan 22, 2025 @ 23:14:44.854	file.path C:\Users\jamesmurphy\AppData\Local\Temp\c0thhy2o.dll @timestamp Jan 22, 2025 @ 23:14:44.854 ag
Jan 22, 2025 @ 23:14:44.269	file.path C:\Users\jamesmurphy\AppData\Local\Temp\c0thhy2o.dll @timestamp Jan 22, 2025 @ 23:14:44.269 ag
Jan 22, 2025 @ 22:11:16.861	file.path C:\Users\jamesmurphy\AppData\Local\Temp\kernel32.dll @timestamp Jan 22, 2025 @ 22:11:16.861 ag
Jan 22, 2025 @ 22:11:15.606	file.path C:\Users\jamesmurphy\AppData\Local\Temp\ukt2cbbt.dll @timestamp Jan 22, 2025 @ 22:11:15.606 ag
Jan 22, 2025 @ 22:11:14.823	file.path C:\Users\jamesmurphy\AppData\Local\Temp\ukt2cbbt.dll @timestamp Jan 22, 2025 @ 22:11:14.823 ag
Jan 22, 2025 @ 21:34:17.152	file.path C:\Users\jamesmurphy\AppData\Local\Temp\kernel32.dll @timestamp Jan 22, 2025 @ 21:34:17.152 ag
Jan 22, 2025 @ 21:34:15.777	file.path C:\Users\jamesmurphy\AppData\Local\Temp\22mm3vut.dll @timestamp Jan 22, 2025 @ 21:34:15.777 ag
Jan 22, 2025 @ 21:34:14.846	file.path C:\Users\jamesmurphy\AppData\Local\Temp\22mm3vut.dll @timestamp Jan 22, 2025 @ 21:34:14.846 ag

Figure 3. Any version of AppData\Local\Temp folder with .DLL files.

The authors found all the logs have Event code '11', the DLLs executed from 'AppData\Local\Temp' folder. Expanding the log, displays the file path and file directory as displayed in Figure 4.

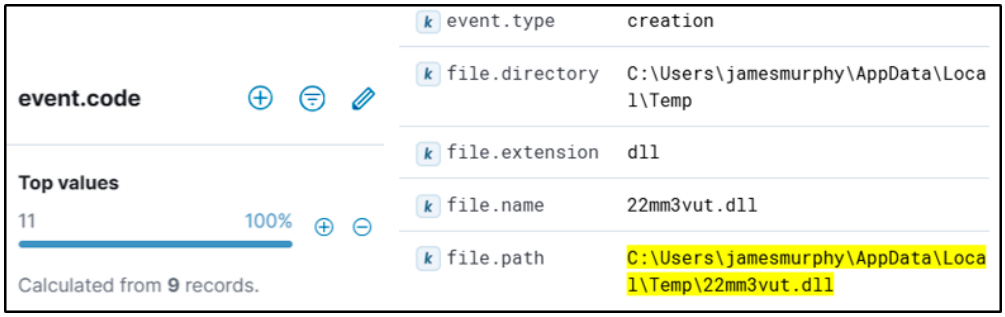


Figure 4. Searching for a File and Location.

To find contextual information like process names and IDs responsible for these hits, the authors added Event Code, File Path, Process Name, and Process ID to the SIEM dashboard as illustrated in Figure 5.

@timestamp	event.code	file.path	process.name	process.pid
Jan 22, 2025 @ 21:34:14.846	11	C:\Users\jamesmurphy\AppData\Local\Temp\22mm3vut.dll	powershell.exe	11,292
Jan 22, 2025 @ 21:34:15.777	11	C:\Users\jamesmurphy\AppData\Local\Temp\22mm3vut.dll	csc.exe	1,288
Jan 22, 2025 @ 21:34:17.152	11	C:\Users\jamesmurphy\AppData\Local\Temp\kernel32.dll	powershell.exe	11,292
Jan 22, 2025 @ 22:11:14.823	11	C:\Users\jamesmurphy\AppData\Local\Temp\ukt2cbbt.dll	powershell.exe	3,132
Jan 22, 2025 @ 22:11:15.606	11	C:\Users\jamesmurphy\AppData\Local\Temp\ukt2cbbt.dll	csc.exe	5,040
Jan 22, 2025 @ 22:11:16.861	11	C:\Users\jamesmurphy\AppData\Local\Temp\kernel32.dll	powershell.exe	3,132
Jan 22, 2025 @ 23:14:44.269	11	C:\Users\jamesmurphy\AppData\Local\Temp\c0thhy2o.dll	powershell.exe	4,788
Jan 22, 2025 @ 23:14:44.854	11	C:\Users\jamesmurphy\AppData\Local\Temp\c0thhy2o.dll	csc.exe	10,624
Jan 22, 2025 @ 23:14:46.589	11	C:\Users\jamesmurphy\AppData\Local\Temp\kernel32.dll	powershell.exe	4,788

Figure 5. Contextual Information Gathered.

To find the unique file names dropped in the AppData folder, the 'file name' visualization reveals four DLLs involved in command line arguments. When filtering the process command line field for the four identified DLL names, the search in Figure 6 returned 16 matches. Each match corresponded to Sysmon Event ID 1 (Process Creation), which aligns with Windows Event ID 4688 in native Windows Security Logs. Both events indicate that a new process was created, confirming that these DLLs were executed on the system.

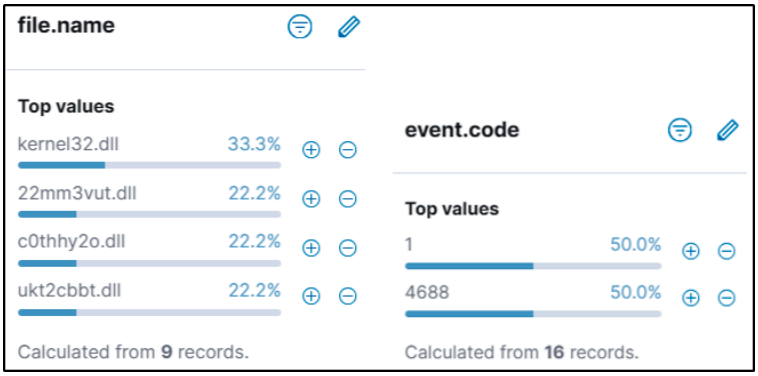


Figure 6. Visualizing DLLs & associated Event Codes.

The SIEM dashboard was updated to include additional fields—Process Command Line, Process Parent Command Line, Process Name, Process ID, and Parent Process ID—to provide deeper context during analysis. Figure 7 displays 8 hits with 'PowerShell.exe' as the parent process and 'Rundll32.exe' as the child process. This shows 'Rundll32' is calling the same DLL found in the initial search query. The parent command line contains a Base64-encoded string, which is used to obscure the actual instructions being executed.

process.command_line	process.parent.name	process.parent.command_line	process.pid	process.parent.pid
"C:\Windows\system32\rundll32.exe" C:\U...	powershell.exe	powershell -encodedcommand KABOAGUAdwAtAE8AYgBqAGUAYwB0ACAA...	7,992	3,132
"C:\Windows\system32\rundll32.exe" C:\U...	powershell.exe	powershell -encodedcommand KABOAGUAdwAtAE8AYgBqAGUAYwB0ACAA...	2,816	11,292
"C:\Windows\system32\rundll32.exe" C:\U...	powershell.exe	powershell -encodedcommand KABOAGUAdwAtAE8AYgBqAGUAYwB0ACAA...	10,676	4,788
rundll32.exe C:\Users\Jamesmurphy\AppDa...	cmd.exe	"C:\Windows\System32\cmd.exe" /c rundll32.exe C:\Users\Jame...	10,888	10,796
"C:\Windows\System32\cmd.exe" /c rundll...	explorer.exe	C:\Windows\Explorer.EXE	10,796	8,560
reg add "HKLM\Software\Microsoft\Window...	cmd.exe	cmd.exe	2,164	9,048
rundll32.exe C:\Users\Jamesmurphy\AppDa...	cmd.exe	"C:\Windows\System32\cmd.exe" /c rundll32.exe C:\Users\Jame...	6,500	2,328
"C:\Windows\System32\cmd.exe" /c rundll...	explorer.exe	C:\Windows\Explorer.EXE	2,328	6,468

Figure 7. Investigating Parent & Child Process for Event Code 1.

Upon reviewing additional logs, this DLL is mentioned in another process create event for modifying registry keys, calling 'reg.exe'. Figure 8 presents the process command line, which displays Process ID '2164' changing the current version run registry key to add key name 'AutoUpdate'. This exhibits similar behavior to other processes that create events with 'RunDll32.dll', which calls 'Kernel32.dll'. This comes across as someone trying to gain persistence.

# process.args_count	10
k process.command_line	reg add "HKLM\Software\Microsoft\Windows\CurrentVersion\Run" /v "AutoUpdate" /t REG_SZ /d "cmd.exe /c rundll32.exe C:\Users\Jamesmurphy\AppData\Local\Temp\kernel32.dll,DllMain" /f
k process.entity_id	{f0d7c56e-a017-65b2-4101-000000007100}
k process.executable	C:\Windows\System32\reg.exe

Figure 8. Process Command Line modifying the Registry Key.

To validate the call to 'Reg.exe' was successful, the authors performed a pivot query to search for Event Code '13', process name 'reg.exe' and process.pid '2164'. Figure 9 presents only one hit, but this confirms the suspicion that the registry key modification was successful (Event Code 13).

event.code	process.name	registry.path	registry.data.strings	process.pid
13	reg.exe	HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run\AutoUpdate	cmd.exe /c rundll32.exe C:\Users\Jamesmurphy\AppData\Local\Temp\kernel32.dll,DllMain	2,164

Figure 9. Reg' Call was successful.

To investigate the file path's involvement in registry key modification, the authors performed a case-insensitive Lucene query searching for 'AppData\Local\Temp' and 'Kernel32.dl', looking for upper and lowercase letters as combinations. With 175 hits, Figure 10 visualizes the top Event Code '4663', which deals with access being granted, but this is contextual and good information to gather, rather than positive actionable intelligence.

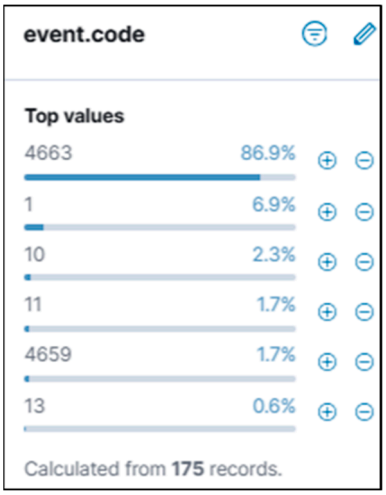


Figure 10. Visualizing Event Codes for Lucene Query.

Removing Event Code '4663' and visualizing the remaining codes, which reveals 23 hits. In terms of Windows Sysmon monitoring, Event Code '1' (process create). '11' (file creation), '13' (Registry Key modification), and others, including '10', refers to a 'process access event. This means that a process has attempted to open another process on the system, often to read or write to its memory space. Adding the Event Code '10' as a filter, Figure 11 illustrates four hits with 'Rundll32.exe' being the source as well as the target image. This indicates no changes were made, and thus this is not suspicious.

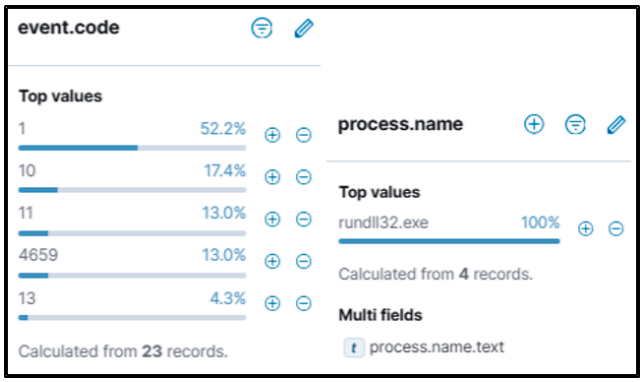


Figure 11. Visualized Top Event Codes and Process Names.

4.2. Threat Hunt #2 Autorun Registry Key Modification

Autorun or the ASEP (AutoStart Extension Point) Registry Keys control the launch for the user when they initially log in. Attackers achieve persistence by modifying these specific keys—an example would be located at 'HKCU\Software\Microsoft\Windows\

CurrentVersion\Run'. This attack focuses on the adversary's attempt to make these modifications and attain persistence on an exploited machine. This allows a binary to be executed with elevated privileges as an example, by the abuse of this normal system behavior. Malware and attackers accomplish this by either placing a program in the startup folder or by calling it via a Registry run key. Placing an entry in the 'run keys' in the Registry or the startup folder results in the called program running on a user logon. When conducted maliciously, the key name or value often appears suspicious, for example, using randomly generated names or referencing executables located in temporary or public folders.

The most straightforward way to simulate the expected behavior is to run the command and check its output as 'reg ADD HKLM\SOFTWARE\\Microsoft\Windows\CurrentVersion\Run/V "Test Entry"/t REG_SZ/F/D "C:\Windows\System32\cmd.exe"', which might succeed on occasion. However, the author designed the query logic for detecting these attacks using the Elasticsearch SIEM as displayed in Table 6. This threat hunt logic identifies when an entry is added into the 'run keys' in the Registry, signalling that an adversary is potentially trying to achieve persistence. This technique is intended to configure the device to execute a program upon startup or even potentially gain escalated privileges. To validate this activity, the associated logs should be reviewed to confirm that an entry was added to the Version\Run registry key.

Table 6. Query Logic for Threat Hunt #2.

Selection	Field	Value
Registry path primary (ANY)	registry_path	*\CurrentVersion\Run, *\Windows\Run, *\Windows\Run, *\WOW6432Node
Registry path extended (ANY)	registry_path	*\Explorer\User Shell Folders, *\Winlogon\Userinit, *\Winlogon\Shell, *\AppInit_DLLs, *\Windows\Load, *\Policies\Explorer\Run, *\Microsoft Visual Studio\Installer\, *\ClickToRun\OfficeClickToRun.exe,
False positive registry values (ANY)	registry_key_name	*\Winlogon\Shell, *\Microsoft\OneDrive\, \SearchIndexer.exe, *\Microsoft\Teams\, *\Microsoft\Edge\Application\

The authors present the abstract code in Table 7 for this query logic to detect potential persistence attempts in the registry. The logic consists of two main conditions: the inclusion of search results containing specific registry paths commonly associated with auto-start execution locations, such as Run, Userinit, Shell, AppInit_DLLs, and Winlogon. Attackers exploit these paths to maintain persistence on a compromised system. The query searches within registry paths and event log fields like WinLog. EventData.ObjectName and registry_path. The second condition acts as an exclusion filter to remove benign or expected entries, such as processes related to Microsoft Visual Studio Installer, OneDrive, Teams, Edge, and SearchIndexer.exe, ensuring that false positives are minimized. The query is designed for security monitoring and threat detection, likely within an endpoint detection and response (EDR) or SIEM system.

Figure 12 displays one hit with contextual details with Event Code 13, Process Name 'reg.exe', Registry path, and Data strings along with Process ID '2164'. This is the same event that was used to confirm the first threat hunt, that 'reg.exe' was successful.

Following Child process ID (2164) and Process Name (reg.exe), a pivot query for process name (reg.exe) and process command line (kernel32) or the process parent.name (reg.exe) and process parent command line (kernel32) displays one hit as illustrated in Figure 13. Here, the child process is 'reg.exe' and the parent process is 'cmd.exe'.

Table 7. Code for detecting potential persistence in registry keys.

Begin Query

Condition 1: Include Search Results where any of the following Registry Path Patterns Match:
- *\CurrentVersion → Run
- *\Windows → Run
- *\Windows → Run
- *\WOW6432Node → Run
- *\Explorer → User Shell Folders
- *\Winlogon → Userinit
- *\Winlogon → Shell
- *\AppInit_DLLs
- *\Windows → Load
- *\Policies → Explorer → Run
- *\Microsoft Visual Studio → Installer
- *\ClickToRun → OfficeClickToRun.exe
- *\Winlogon → Shell
- *\Microsoft → OneDrive
- \SearchIndexer.exe
- *\Microsoft → Teams
- *\Microsoft → Edge → Application

Search In:
- registry.path
- winlog.event_data.ObjectName
- winlog.event_data.TargetObject
- registry_path

Condition 2: Exclude Results where any of the following Processes or Registry Paths Match:
- Microsoft Visual Studio → Installer → ClickToRun → OfficeClickToRun.exe
- *\CurrentVersion → Winlogon → Shell
- Microsoft → OneDrive
- Windows → System32 → SearchIndexer.exe
- Microsoft → Teams
- Microsoft → Edge → Application

Search In:
- winlog.event_data.ProcessName
- process.executable
- registry_key_name
- process_path

End Query

event.code	process.name	registry.path	registry.data.strings	process.pid
13	reg.exe	HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run\AutoUpdate	cmd.exe /c rundll32.exe C:\Users\Jamesmurphy\AppData\Local\Temp\kernel32.dll,DllM	2,164

Figure 12. Registry Key Search.

The authors performed a pivot query with child process as 'cmd.exe' with child process ID '9048' or the process parent name as 'cmd.exe' with process parent ID '9048'. This reveals six hits as illustrated in Figure 14, the first row has 'cmd.exe' as the child process and 'notepad.exe' as the parent process. The second row as 'reg.exe' as the child process and 'cmd.exe' as the parent process. Then, from the same parent process ID (9048), a command line child process 'cmd.exe/c.1.bat' is spawned.

process.name	process.command_line	process.parent.command_line	process.parent.name	process.pid	process.parent.pid
reg.exe	reg add "HKLM\Software\Microsoft\Windows\CurrentVersion\Run" /v "AutoUpdate" /t REG_SZ /d "cmd.exe /c rundll32.exe C:\Users\Jamesmurphy\AppData\Local\Temp\kernel32.dll,DllMain" /f	cmd.exe	cmd.exe	2,164	9,048

Figure 13. Filter for Process Name and Process ID.

process.name	process.command_line	process.parent.command_line	process.parent.name	process.pid	process.parent.pid
cmd.exe	cmd.exe	notepad.exe	notepad.exe	9,048	2,356
reg.exe	reg add "HKLM\Software\Micro…	cmd.exe	cmd.exe	2,164	9,048
cmd.exe	-	-	-	9,048	-
cmd.exe	cmd.exe /c 1.bat	cmd.exe	cmd.exe	4,884	9,048
cmd.exe	-	-	-	9,048	-
cmd.exe	-	-	-	9,048	-

Figure 14. Query for Process 'cmd.exe' with parent and child process ID '9048'.

Now performing a third pivot with process name 'notepad.exe' with process ID '2356' or the process parent name 'notepad.exe' and process parent ID '2356', Figure 15 reveals 3 hits with the first row having 'notepad.exe' as the child process with a randomly named executable (QazTOWNse.exe) as the parent process with Parent ID '3452'. Then 'Notepad.exe' spawns 'cmd.exe' in the second row and Event Code '4689' for Windows Native Loging for process termination.

event.code	process.name	process.command_line	process.parent.command_line	process.parent.name	process.pid	process.parent.pid
1	notepad.exe	notepad.exe	"C:\Users\JAMESM~1\AppData\Local…	QazTOWNse.exe	2,356	3,452
1	cmd.exe	cmd.exe	notepad.exe	notepad.exe	9,048	2,356
4689	notepad.exe	-	-	-	2,356	-

Figure 15. Query for Process 'notepad.exe' with parent and child process ID '2356'.

Pivoting backwards to investigate 'QazTOWNse.exe', the authors performed another pivot to search for this executable as a child and parent process and with process ID '3452'. This reveals 546 hits, which are visualized with Event Code '10', representing 99.1% of the data, as shown in Figure 16.

The authors filtered for Event Code '10' and process name 'QazTOWNse.exe' with Winlog Events (Source User, Target Image, Target Username). The query returned 541 hits in total. Figure 17 illustrates a representative subset of these results for clarity. Browsing these logs reveals that 'QazTOWNse.exe' is injecting itself into different processes as the source user (jamesmurphy), while the target user is different, which is highly suspicious.

Focusing on the process name 'notepad.exe' or Winlog event data SourceImage as 'notepad.exe' or the Winlog event data TargetImage as 'notepad.exe', displays 433 hits, which are filtered to keep only the randomly named executables. Visualizing now reveals 81 hits as depicted in Figure 18, which shows the executables targeting 'notepad.exe'. Figure 16. Visualization of process interactions involving notepad.exe as either the source or target process, filtered to highlight randomly named executables targeting it, indicating potential process injection activity. This illustrates the mapping of processes where notepad.exe appears as either the initiating (source) or receiving (target) process in Sysmon event logs. For readers unfamiliar with Sysmon data structures, each node represents a process, while connecting lines show execution or injection relationships. The highlighted

81 instances correspond to suspicious executables with non-standard names, a standard indicator of malicious code injection. In the context of intrusion detection, such relationships are significant because benign processes, such as notepad.exe, are rarely designed to interact with or be targeted by unfamiliar executables. These anomalies often occur when an attacker injects code into a trusted process to evade security tools and inherit higher privileges. By correlating parent-child process relationships, security analysts can trace execution chains, identify injected processes, and pinpoint potential compromise points in the attack lifecycle.

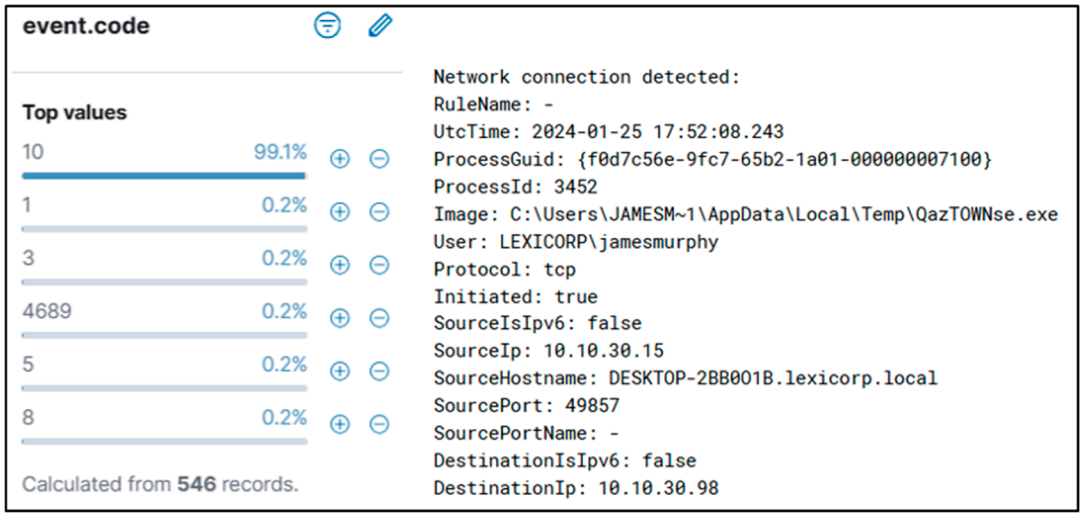
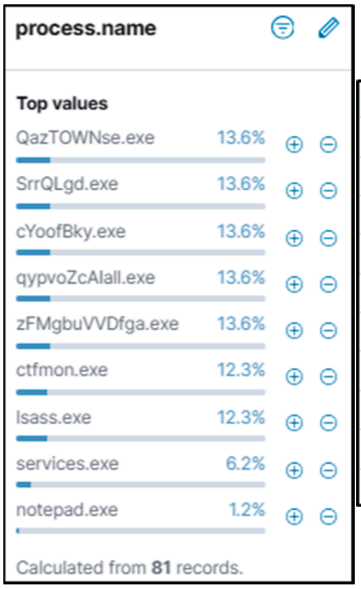


Figure 16. Visualization and Message information for Process 'QazTOWNse.exe'.

event.code	process.name	winlog.event_data.SourceUser	winlog.event_data.TargetImage	winlog.event_data.TargetU...	process.pid
10	QazTOWNse.exe	LEXICORP\jamesmurphy	C:\Windows\system32\notepad.exe	LEXICORP\jamesmurphy	3,452
10	QazTOWNse.exe	LEXICORP\jamesmurphy	C:\Windows\System32\smss.exe	NT AUTHORITY\SYSTEM	3,452
10	QazTOWNse.exe	LEXICORP\jamesmurphy	C:\Windows\System32\smss.exe	NT AUTHORITY\SYSTEM	3,452
10	QazTOWNse.exe	LEXICORP\jamesmurphy	C:\Windows\system32\csrss.exe	NT AUTHORITY\SYSTEM	3,452
10	QazTOWNse.exe	LEXICORP\jamesmurphy	C:\Windows\system32\csrss.exe	NT AUTHORITY\SYSTEM	3,452

Figure 17. Filter for process name 'QazTOWNse.exe' and Winlog Events.



process.name	winlog.event_data.TargetImage	process.pid
services.exe	C:\Windows\system32\notepad.exe	924
lsass.exe	C:\Windows\system32\notepad.exe	964
lsass.exe	C:\Windows\system32\notepad.exe	964
cYoofBky.exe	C:\Windows\system32\notepad.exe	5,868
cYoofBky.exe	C:\Windows\system32\notepad.exe	5,868
cYoofBky.exe	C:\Windows\system32\notepad.exe	5,868
cYoofBky.exe	C:\Windows\system32\notepad.exe	5,868
cYoofBky.exe	C:\Windows\system32\notepad.exe	5,868
cYoofBky.exe	C:\Windows\system32\notepad.exe	5,868
cYoofBky.exe	C:\Windows\System32\notepad.exe	5,868

Figure 18. Visualizing process name 'notepad.exe' and Source and Target names.

Figure 19 displays the process name 'cYoofBky.exe' with source and target user as 'jamesmurphy'. After a while, the binary executable reveals the same source user. The target user (NT Authority\System) is the 'process injection' with the malicious executable. This fills into the memory space of 'notepad.exe', allowing 'nptepad.exe' to be a conduit with System admin level privileges. This provides a detailed example of privilege escalation through process injection, captured in Sysmon logs. Here, the executable cYoofBky.exe—a randomly named and suspicious binary—injects itself into the memory space of notepad.exe. The Sysmon event data records a change in target user context from a regular domain account (jamesmurphy) to the high-privilege NT AUTHORITY\SYSTEM account. For readers less familiar with Sysmon data structures, such a privilege shift is a critical red flag, as it indicates that an attacker-controlled process has successfully hijacked a trusted application to execute commands with elevated rights. This tactic allows adversaries to bypass User Account Control (UAC) and maintain stealth, as the injected process inherits the legitimacy of the host application. Documenting these privilege transitions in the process tree not only aids in real-time detection but also provides forensic evidence for incident response teams to reconstruct the intrusion path.

process.name	winlog.event_data.TargetImage	winlog.event_data.SourceUser	winlog.event_data.TargetUser
cYoofBky.exe	C:\Windows\system32\notepad.exe	LEXICORP\jamesmurphy	LEXICORP\jamesmurphy
cYoofBky.exe	C:\Windows\system32\notepad.exe	LEXICORP\jamesmurphy	LEXICORP\jamesmurphy
cYoofBky.exe	C:\Windows\system32\notepad.exe	LEXICORP\jamesmurphy	LEXICORP\jamesmurphy
cYoofBky.exe	C:\Windows\system32\notepad.exe	LEXICORP\jamesmurphy	LEXICORP\jamesmurphy
cYoofBky.exe	C:\Windows\System32\notepad.exe	LEXICORP\jamesmurphy	NT AUTHORITY\SYSTEM
cYoofBky.exe	C:\Windows\System32\notepad.exe	LEXICORP\jamesmurphy	NT AUTHORITY\SYSTEM

Figure 19. Process Injection.

When adversaries inject code into a process running at a higher privilege level, they execute code or executables from the new elevated process being controlled by them. Filtering for child and parent process name 'cmd.exe' and the child and process command line '1.bat', Figure 20 illustrates 18 hits with 'cmd.exe' as the child process starting the parent process 'cmd.exe' with arguments for the batch file (1.bat) as the Process ID '4884' becomes the parent process ID. The command line arguments included in the batch file spawn various commands (ipconfig, ping, hostname, net) with a 'timeout' for a 30-s break when executing the batch commands. By correlating these activities from the Event logs, it clearly indicates discovery being performed via a batch file.

process.name	process.command_line	process.parent.name	process.parent.command_line	process.pid	process.parent.pid
cmd.exe	cmd.exe /c 1.bat	cmd.exe	cmd.exe	4,884	9,048
PING.EXE	ping 127.0.0.2 -n 100	cmd.exe	cmd.exe /c 1.bat	3,012	4,884
timeout.exe	timeout /t 30 /nobreak	cmd.exe	cmd.exe /c 1.bat	2,916	4,884
ipconfig.exe	ipconfig	cmd.exe	cmd.exe /c 1.bat	8,728	4,884
timeout.exe	timeout /t 30 /nobreak	cmd.exe	cmd.exe /c 1.bat	1,288	4,884
HOSTNAME.EXE	hostname	cmd.exe	cmd.exe /c 1.bat	2,956	4,884

Figure 20. Batch file executing discovery commands.

Table 8 provides a concise comparison between the proposed Elasticsearch-based SIEM threat hunting framework and commonly used detection systems. Unlike traditional signature-based IDS tools that rely solely on predefined attack patterns or anomaly-based

IDS that focus primarily on deviations from network traffic baselines, the proposed framework combines hypothesis-driven hunting with granular host-level telemetry from Sysmon and Windows Event Logs. Compared to commercial SIEM platforms, our approach offers a higher degree of customization, enabling analysts to craft precise queries in Kibana or Lucene that target privilege escalation and persistence techniques at the process chain level. Furthermore, the integration of statistical validation, utilizing Z-score anomaly detection alongside confusion matrix-based performance metrics, strengthens the framework's reliability in identifying zero-day or stealthy threats. This combination of adaptable detection logic, fine-grained process correlation, and quantitative evaluation underscores the system's novelty and added value in enterprise security operations.

Table 8. Comparative Analysis of Proposed System with Existing Detection Systems.

Feature	Signature-Based	Anomaly-Based	Proposed SIEM Solution
Detection approach	Pattern matching	Baseline-deviation	Event correlation + hypothesis-driven hunts using Sysmon & Windows logs
Data sources	Network traffic	Host/Network metrics	Host-level Sysmon events, Windows Event Logs, registry/file system monitoring
Zero-Day Detection	Low	Moderate	High
False-Positive Control	Low-Moderate	Low	High
Use Case Suitability	Known attacks in network traffic	Unknown network anomalies	Privilege escalation & persistence detection via registry/DLL analysis

To operate the proposed detection logic in a real-world enterprise SIEM environment, several deployment considerations must be addressed. First, the SIEM must be integrated with host-level telemetry sources, such as Sysmon and native Windows Event Logs, across all monitored endpoints. This requires deploying endpoint agents or utilizing centralized log forwarding mechanisms with appropriate bandwidth and storage capacity to handle large volumes of event data. Scalability can be achieved by using distributed Elasticsearch clusters, which enable horizontal expansion as log ingestion rates increase. Role-based access control (RBAC) and data segregation should be configured to align with organizational security policies. The detection logic, including KQL and Lucene queries, can be encapsulated as reusable SIEM rules or dashboards, allowing security analysts to trigger automated alerts when matching patterns occur. To reduce false positives, periodic tuning and validation of detection rules are essential, incorporating feedback from incident response teams. Limitations include potential performance overhead on endpoints due to verbose logging, and the need for continuous signature or baseline updates to adapt to evolving attack techniques. Moreover, deployment success depends on trained SOC analysts capable of interpreting correlated process and registry data, ensuring that detections are triaged effectively without alert fatigue in high-volume environments.

5. Results Obtained

5.1. Summary of the First Threat Hunt

The authors searched for a DLL that matched the WhisperGate schema being dropped in the '\AppData\Local\Temp' folder. The authors identified potential malicious executions as Sysmon Event Code 1 for Process creation and Windows Event '4688'. This revealed PowerShell being called by 'Rundll32.exe' and then 'Reg.exe', trying to add and perform a Windows Run Registry key modification. The authors also confirmed that the registry key was successfully modified with Sysmon Event ID '13'. A wildcard search revealed the file path existed in multiple event codes. If suspect activity is identified, the Cyber analyst

can proceed to remove the potentially malicious entry found in the registry to prevent any further escalation or damage from the event, or respond according to internal process if removal is not possible. Malicious programs are often dropped within the Windows startup folder, and investigating that path can be part of the triage strategy. Confirmation of a file being malicious can be accomplished by examining run keys that point to an unusual binary (such as abcdefg.exe), unusual folder paths (such as temp/downloads), or if there is any other suspicion that warrants inspection; additional analysis to determine capability and intention is advised.

5.2. Summary of the Second Threat Hunt

This hunt started by searching for registry keys that are often abused for persistence and privilege escalation. 'Kernel32.dll' performed registry key addition. Looking at different parent and child process names and process IDs, moving from 'cmd.exe' to 'notepad.exe' to 'QazTOWNse.exe' with Winlog events. This revealed the root cause of the attack, which was performed using a batch file. Parsing of logs for events appended with WinEvent ID 4657 (Registry Value Modification) or other applicable registry-related events is recommended to ascertain the timeline of the registry being modified. This will enable the analyst to comprehend how/when the registry key was altered and assist in investigating any activity occurring immediately after the alerted event for further signs of malicious activity. The startup program being executed will be in harmony with other malicious behavior. Thus, further review is warranted into additional evidence of compromise, such as behavior indicative of domain enumeration, privilege escalation, or lateral movement.

Once the attack is verified, analysts need to investigate the root cause by tracing the associated process that created the registry key backward through the process chain to determine what originally triggered the creation of the registry key. In some cases, the process chain may not shed much light as the process is newly executed, in this case, reviewing Service creations (WinEvent 7045 for example), zip/rar extractions, LNK executions, new execution/opening of Microsoft Office applications such as Word or Outlook, before the registry key creation may help shed light on what the root cause may be. Analysts should also investigate process executions related to the registry value observed in the registry creation. In many cases, attackers or malware will also create Service persistence (WinEvent 7045), which will point to the same binary found in the registry key. Looking for excessive executions of the applicable binary found in the registry key point to Service creation, if it was not logged as being newly created. Identifying what the binary performed on the host is crucial to determining the scope and triaging what the binary changed, modified, or accessed on the host, such as whether it performed credential dumping or other malicious activities.

Table 9 provides a comparative summary of two distinct threat hunts conducted to detect and analyze potential adversary techniques. The first hunt focused on identifying malicious DLL execution linked to WhisperGate, while the second hunt targeted registry key abuses for persistence and privilege escalation. This highlights key indicators, detection methods, findings, and recommended actions for both threat hunt cases.

Table 10 presents a quantitative breakdown of key elements identified during the two threat hunts. It highlights the number of analyzed events, identified processes, suspicious registry modifications, detected malicious file paths, and recommended investigation areas. These numerical insights help in understanding the scale and complexity of each threat hunt.

Figure 21 provides a comparative analysis of key security metrics from two distinct threat hunts. The first hunt focused on detecting malicious DLL execution related to WhisperGate, while the second targeted registry key abuse for persistence and privilege escalation. The graph quantifies critical parameters, including the number of security

events analyzed, identified processes, registry modifications, suspicious file paths, and investigative focal points. This visualization offers cybersecurity professionals and corporate security teams a structured perspective on the scale and complexity of each threat hunt, aiding in data-driven decision-making for incident response and threat mitigation.

Table 9. Comparative Summary for Threat Hunts.

Aspect	First Threat Hunt	Second Threat Hunt
Focus	Detection of WhisperGate DLL in \AppData\Local\Temp	Abuse of registry keys for persistence and privilege escalation
Key Indicators	- DLL file matching WhisperGate schema- Sysmon Event Code 1 (Process creation)- Windows Event 4688 (PowerShell execution via Rundll32.exe)- Registry modification detected via Sysmon Event ID 13	- Registry key modification via Kernel32.dll- Parent/child process analysis (cmd.exe → notepad.exe → QazTOWNse.exe)- WinEvent ID 4657 (Registry modification tracking)
Detection Methods	- Wildcard search for file path in event logs- Run key analysis for unusual binaries or paths	- Parsing logs for registry changes- Identifying process execution chains leading to registry modification
Key Findings	- Malicious programs placed in Windows startup folder- Registry run keys modified for persistence	- Attack initiated via batch file execution- Process chain analysis reveals execution flow- Service persistence potentially established via WinEvent 7045
Recommended Actions	- Remove malicious registry entries- Investigate startup folder activity- Confirm malicious files via additional analysis	- Trace root cause by analyzing process execution history- Investigate related Service creations, document activity, or email-based threats- Conduct environment-wide artifact search

Table 10. Quantitative breakdown of Threat Hunts.

Aspect	First Threat Hunt	Second Threat Hunt
Key Events Analyzed	3 (Sysmon Event 1, Windows Event 4688, Sysmon Event 13)	2 (WinEvent ID 4657, WinEvent 7045)
Processes Identified	3 (Rundll32.exe, PowerShell.exe, Reg.exe)	3 (cmd.exe, notepad.exe, QazTOWNse.exe)
Suspicious Registry Modifications	1 (Windows Run Registry key)	Multiple (Persistence and privilege escalation keys)
Potential Malicious File Paths Detected	2 (\AppData\Local\Temp\, Windows Startup folder)	1 (Registry modification path)
Recommended Investigation Areas	3 (Run keys, startup folder, suspicious binaries)	4 (Process execution chains, service creations, registry modifications, related artifacts)

After the artifacts have been collected, analysts search across the environment for the same or similar artifacts to determine the potential scope of the attack/compromise. Additional indications that may be of assistance are document names, email subjects, folder paths and service names if any of these are identified. These artifacts often follow an obvious schema or format, making pivoting and scoping easier.

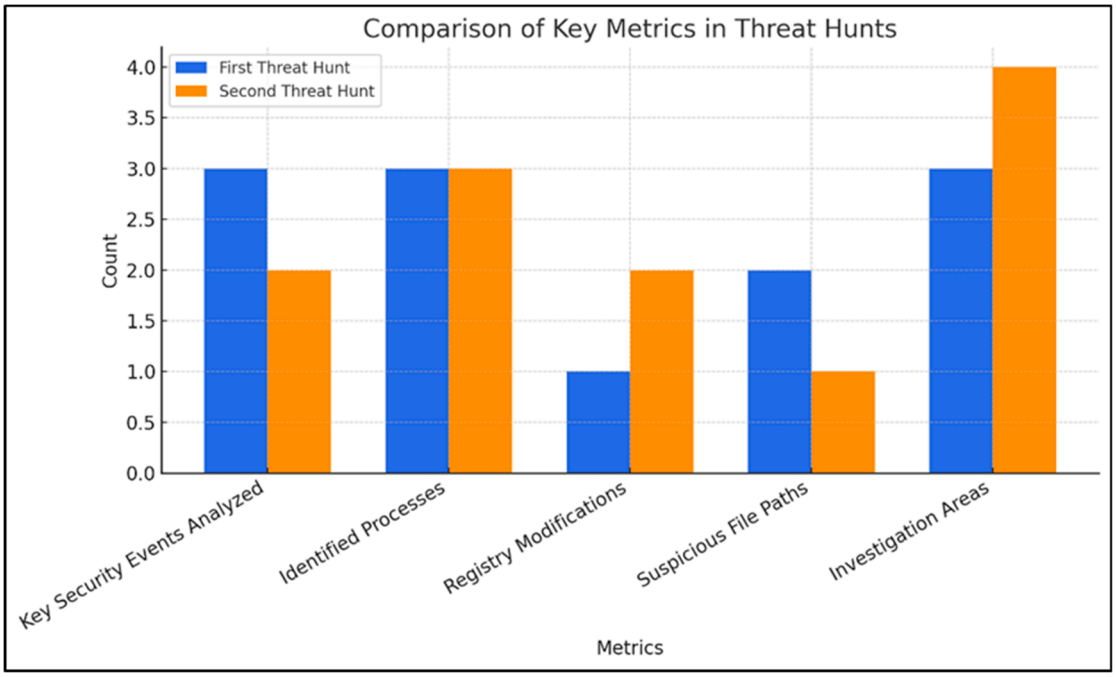


Figure 21. Graphical Representation of the analysis.

6. Mathematical Validation

The authors validated the threat hunt research using two mathematical options:

i. Detection Accuracy Metrics with Confusion Matrix Analysis

The objective of using Confusion Matrix Analysis is to quantitatively evaluate the detection model's effectiveness in identifying malicious activities from log data (Sysmon + Windows Events) as displayed in Table 11 with manually verified alerts from logs related to DLL drops, registry modifications which serve as the ground truth.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} = 0.956 \quad (1)$$

$$Precision = \frac{TP}{TP + FP} = 0.876 \quad (2)$$

$$Recall = \frac{TP}{TP + FN} = 0.948 \quad (3)$$

$$F1 - Score = 2 \times \frac{(Precision \times Recall)}{(Precision + Recall)} = 0.911 \quad (4)$$

Table 11. Registry Threat Detection Metrics.

Metric	Value (Registry Key Detection)
TP	128
TN	402
FP	18
FN	7

The approach adopted here is to use the Confusion Matrix and derive standard classification metrics, where TP = True Positives for correctly identified malicious events. TN = True Negatives for correctly identified benign events. FP = False Positives for benign events incorrectly flagged as malicious. FN = False Negatives for missed malicious events.

These results, derived from a confusion matrix, provide a quantitative validation of the threat detection system developed using Elasticsearch SIEM. The analysis is based on

manually verified ground truth from system logs, focusing on registry key modifications and process injection attempts associated with privilege escalation. The confusion matrix yielded 128 true positives (*TP*), 402 true negatives (*TN*), 18 false positives (*FP*), and 7 false negatives (*FN*).

These values produced an accuracy of 95.6%, indicating that the system correctly classified the majority of events. The precision, calculated at 87.6%, reflects the system's ability to minimize false alarms, ensuring that most alerts genuinely correspond to malicious activity. A recall score of 94.8% demonstrates high sensitivity, meaning the system successfully detected the majority of actual threats without missing critical events. The F1-score, a harmonic mean of precision and recall, was 91.1%, confirming that the system maintains a strong balance between correctly identifying threats and minimizing erroneous detections.

The results validate the system's effectiveness in real-time scenarios by leveraging detailed log correlation and behavioral signatures. This mathematical evaluation supports the deployment of such detection mechanisms in enterprise environments, where precision and reliability are crucial for reducing false alerts and swiftly identifying privilege escalation threats.

ii. Statistical Anomaly Detection via Z-Score Method.

The objective here is to validate anomaly detection on event frequency for registry/process-based events, like a spike in Sysmon Event ID 13 for registry modification. The approach is to utilize Z-score normalization to detect outliers in event log frequency over time. The Z-score method proves effective in modeling expected behavior and flagging sudden, rare activities like registry modifications or process injections. This complements rule-based SIEM logic with statistical anomaly detection. This statistical method helps highlight spikes that correlate with potential attacks as presented in Table 12 where:

x_i = Observed count of a specific event at time i

μ = Mean event count

σ = Standard Deviation of event count

Any $|Z_i| > 2$ or > 3 is considered anomalous depending on the strictness, so the Z-Score normalization is calculated as

$$Z_i = \frac{x_i - \mu}{\sigma} \tag{5}$$

Table 12. Registry Modification Events (Event ID 13).

Time (HH:MM)	Count (x_i)	Mean (μ)	Std. Dev (σ)	Z-Score (Z_i)	Anomaly
09:00	3	1.1	0.8	2.37	Present
10:00	1	1.1	0.8	−0.12	Not present
11:00	0	1.1	0.8	−1.37	Not present
12:00	4	1.1	0.8	3.62	Present

The results of statistical anomaly detection using the Z-score method demonstrated the effectiveness of identifying unusual activity patterns associated with privilege escalation attempts. By analyzing event frequency distributions, particularly focusing on Sysmon Event ID 13 (registry key modification), the Z-score was used to normalize deviations from the mean across different time intervals. Events with Z-scores exceeding the threshold of ± 2 were flagged as anomalous, indicating a significant deviation from the norm. In one instance, a Z-score of 3.62 was observed at 12:00, corresponding with a registry key change that later correlated with a confirmed malware activity, specifically the injection of a malicious DLL. Similarly, another Z-score of 2.37 at 09:00 pointed to a previously undocumented registry modification, which, upon deeper inspection, was linked to a batch execution script responsible for persistence tactics. The low Z-scores recorded during

non-attack periods reaffirmed the reliability of this model in filtering routine operations from potential threats. Plotting Z-scores over time reveals anomalous behavior aligned with confirmed attacks.

Figure 22 presents the mathematical validation of the threat detection capabilities discussed in the research. The first graph, on the left, represents the performance of the detection system using standard classification metrics derived from confusion matrix analysis. This includes Accuracy, Precision, Recall, and F1-Score, which provide a quantitative evaluation of the system's ability to correctly identify malicious events such as unauthorized registry modifications or DLL executions. With an accuracy of 95.6%, the system demonstrates reliable performance in distinguishing between malicious and benign activity. A precision of 87.6% indicates that most flagged events were truly malicious, while a recall of 94.8% shows that the system successfully detected nearly all malicious instances. The F1-Score, balancing both precision and recall, stands at 91.1%, suggesting a robust and consistent detection capability suitable for deployment in high-risk enterprise environments.

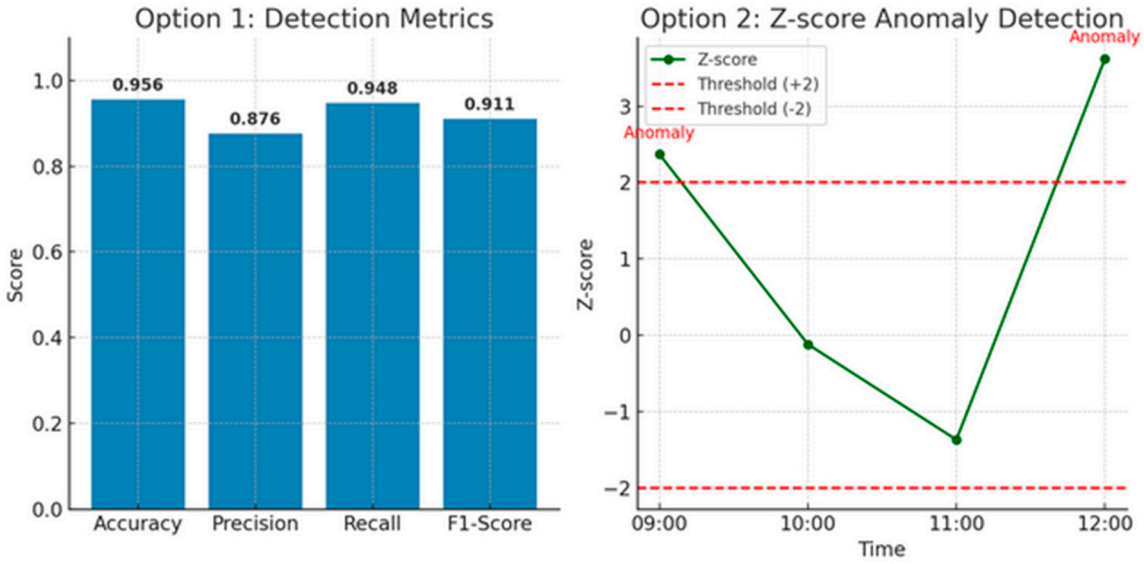


Figure 22. Visualizing the Validation Options.

The second graph, on the right, illustrates the application of statistical anomaly detection using Z-score analysis. This time-series plot tracks the Z-scores of registry modification events over four different time intervals. Spikes that exceed a threshold of ± 2 are marked in red as anomalies, aligning with confirmed instances of privilege escalation attempts. For example, the spikes at 09:00 and 12:00 with Z-scores of 2.37 and 3.62, respectively, indicate statistically significant deviations from normal system behavior, corresponding to suspicious registry activity. This graph highlights the effectiveness of statistical models in identifying outliers in event frequency, which may signal zero-day threats or stealthy persistence mechanisms. Together, these graphs validate the precision and depth of the threat detection methodology.

Overall, the results underscore the potential of integrating mathematical anomaly detection models alongside SIEM platforms like Elasticsearch to improve precision in detecting stealthy and evolving privilege escalation techniques.

7. Conclusions

This research highlights the crucial role of continuous monitoring, robust logging infrastructure, and sophisticated threat intelligence in proactively detecting, investigating, and responding to advanced cyber threats. The second scenario examined registry key

abuse for persistence and privilege escalation. By analyzing process relationships and Winlog events, the research identified a chain of events originating from a batch file, leading to the execution of suspicious processes. This scenario highlights the importance of analyzing registry modification events to understand the attack timeline, identify the root cause, and investigate subsequent malicious activities, including domain enumeration, privilege escalation, and lateral movement. The Confusion metrics also affirm the robustness of the threat hunting logic implemented, particularly in filtering out benign registry changes commonly made by legitimate applications such as Microsoft OneDrive or Teams. The Z-score method provides a quantitative foundation for incorporating behavioral baselines into threat hunting strategies. Unlike static rule-based detections, Z-score-based validation enables the detection of new or evolving attack patterns that deviate from historically normal behavior. It also reduces false positives by identifying statistically significant anomalies rather than relying solely on predefined indicators. This research highlights the need for continuous refinement of threat hunting methodologies, incorporating advanced techniques such as machine learning and artificial intelligence, to enhance detection capabilities and stay ahead of the evolving threat landscape.

Author Contributions: Conceptualization, A.B. and S.R.; methodology, A.B., L.S.; software, A.B.; validation, S.R. and A.B.; formal analysis, L.S.; investigation, S.R.; resources, A.B.; data curation, A.B.; writing—original draft preparation, A.B.; writing—review and editing, S.R.; visualization, A.B.; supervision, S.R.; project administration, L.S.; funding acquisition, S.R. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Data Availability Statement: The original contributions presented in this study are included in the article. Further inquiries can be directed to the corresponding authors.

Conflicts of Interest: The authors declare no conflict of interest.

References

1. What is Privilege Escalation? Crowdstrike.Com. 2014. Available online: https://www.crowdstrike.com/en-us/cybersecurity-101/cyberattacks/privilege-escalation/ (accessed on 22 August 2025).
2. Paolomatarazzo. User Account Control—Windows Security. Learn.Microsoft.Com. 2023. Available online: https://learn.microsoft.com/en-us/windows/security/application-security/application-control/user-account-control/ (accessed on 22 August 2025).
3. Lutkevich, B. What Is Access Control List (ACL)? TechTarget. 2022. Available online: https://www.techtarget.com/searchnetworking/definition/access-control-list-ACL (accessed on 22 August 2025).
4. Alvinashcraft. Access Control Lists—Win32 Apps. Learn.Microsoft.Com. 2023. Available online: https://learn.microsoft.com/en-us/windows/win32/secauthz/access-control-lists (accessed on 22 August 2025).
5. NVD—Cve-2016-0189. Nist.Gov. 2016. Available online: https://nvd.nist.gov/vuln/detail/cve-2016-0189 (accessed on 8 February 2025).
6. Lessing, M. Case Study: WannaCry Ransomware. SDxCentral. 2022. Available online: https://www.sdxcentral.com/news/bugpocalypse-now-zombieload-hits-intel-processors-microsoft-warns-of-wannacry-like-worm/ (accessed on 22 August 2025).
7. Process Injection—Threat Detection Report. Red Canary. Available online: https://redcanary.com/threat-detection-report/techniques/process-injection/ (accessed on 22 August 2025).
8. Compean, N. What Is Persistence in Cybersecurity and How Do You… BeyondTrust. 2023. Available online: https://www.beyondtrust.com/blog/entry/what-is-persistence-in-cybersecurity (accessed on 22 August 2025).
9. Boot or Logon Autostart Execution: Registry Run Keys/Startup Folder, Sub-Technique T1547.001—Enterprise | MITRE ATT&CK®. Available online: https://attack.mitre.org/techniques/T1547/001/ (accessed on 22 August 2025).
10. Scheduled Task/Job, Technique T1053—Enterprise | MITRE ATT&CK® .Attack.Mitre.Org. Available online: https://attack.mitre.org/techniques/T1053/ (accessed on 22 August 2025).
11. IBM. Data exfiltration. Ibm.Com. 2023. Available online: https://www.ibm.com/think/topics/data-exfiltration (accessed on 22 August 2025).
12. Mimikatz. GitHub. 2021. Available online: https://github.com/ParrotSec/mimikatz (accessed on 22 August 2025).

13. Cobalt Strike. GitHub. 2025. Available online: https://github.com/cobalt-strike (accessed on 8 February 2025).
14. MITRE. MITRE ATT&CK™. Mitre.Org. 2024. Available online: https://attack.mitre.org/ (accessed on 22 August 2025).
15. Branescu, I.; Grigorescu, O.; Dascalu, M. Automated Mapping of Common Vulnerabilities and Exposures to MITRE ATT&CK Tactics. *Information* **2024**, *15*, 214. [CrossRef]
16. Khanan, A.; Mohamed, Y.A.; Mohamed, A.H.H.M.; Bashir, M. From Bytes to Insights: A Systematic Literature Review on Unraveling IDS Datasets for Enhanced Cybersecurity Understanding. *IEEE Access* **2024**, *12*, 59289–59317. [CrossRef]
17. Pratama, D.; Suryanto, N.; Adiputra, A.A.; Le, T.-T.-H.; Kadiptya, A.Y.; Iqbal, M.; Kim, H. CIPHER: Cybersecurity Intelligent Penetration-Testing Helper for Ethical Researcher. *Sensors* **2024**, *24*, 6878. [CrossRef] [PubMed]
18. Bouom, A.L.P.T.; Lienou, J.-P.; Geh, W.E.; Nelson, F.F.; Shetty, S.; Kamhoua, C. TriAssetRank: Ranking Vulnerabilities, Exploits, and Privileges for Countermeasures Prioritization. *IEEE Trans. Inf. Forensics Secur.* **2024**, *19*, 10189–10205. [CrossRef]
19. Ryu, D.; Lee, S.; Yang, S.; Jeong, J.; Lee, Y.; Shin, D. Enhancing Cybersecurity in Energy IT Infrastructure Through a Layered Defense Approach to Major Malware Threats. *Appl. Sci.* **2024**, *14*, 10342. [CrossRef]
20. He, D.; Gu, H.; Zhu, S.; Chan, S.; Guizani, M. A Comprehensive Detection Method for the Lateral Movement Stage of APT Attacks. *IEEE Internet Things J.* **2024**, *11*, 8440–8447. [CrossRef]
21. Kasri, W.; Himeur, Y.; Alkhazaleh, H.A.; Tarapiah, S.; Atalla, S.; Mansoor, W.; Al-Ahmad, H. From Vulnerability to Defense: The Role of Large Language Models in Enhancing Cybersecurity. *Computation* **2025**, *13*, 30. [CrossRef]
22. Paracha, M.A.; Jamil, S.U.; Shahzad, K.; Khan, M.A.; Rasheed, A. Leveraging AI for Network Threat Detection—A Conceptual Overview. *Electronics* **2024**, *13*, 4611. [CrossRef]
23. Smiliotopoulos, C.; Kambourakis, G.; Kolias, C. Detecting Lateral Movement: A Systematic Survey. *Heliyon* **2024**, *10*, e26317. [CrossRef]
24. Omar, A.H.E.; Soubra, H.; Moulla, D.K.; Abran, A. An Innovative Honeypot Architecture for Detecting and Mitigating Hardware Trojans in IoT Devices. *IoT* **2024**, *5*, 730–755. [CrossRef]
25. Liu, M.; Teng, F.; Zhang, Z.; Ge, P.; Sun, M.; Deng, R.; Cheng, P.; Chen, J. Enhancing Cyber-Resiliency of DER-Based Smart Grid: A Survey. *IEEE Trans. Smart Grid* **2024**, *15*, 4998–5030. [CrossRef]
26. Zhang, Z.; Liu, M.; Sun, M.; Deng, R.; Cheng, P.; Niyato, D.; Chow, M.-Y.; Chen, J. Vulnerability of Machine Learning Approaches Applied in IoT-Based Smart Grid: A Review. *IEEE Internet Things J.* **2024**, *11*, 18951–18975. [CrossRef]
27. Alsharabi, N.; Bhardwaj, A.; Alshammari, T.S.; Alotaibi, S.; Alshammari, D.; Jadi, A. Threat Hunting the Shadows: Detecting Adversary Lateral Movement with Elasticsearch. *IEEE Access* **2025**, *13*, 62341–62352. [CrossRef]
28. Kim, J.; Jo, E.; Lee, S.; Cho, T. A Model for Illegal File Access Tracking Using Windows Logs and Elastic Stack. *J. Inf. Process. Syst.* **2021**, *17*, 772–786. [CrossRef]
29. Getting Started: Use Elastic Security for SIEM | Starting with the Elasticsearch Platform and Its Solutions [8.12] | Elastic. Available online: https://www.elastic.co/guide/en/starting-with-the-elasticsearch-platform-and-its-solutions/current/getting-started-siem-security.html (accessed on 22 August 2025).
30. DUroz; Rodríguez, R.J. Characteristics and detectability of Windows auto-start extensibility points in memory forensics. *Digit. Investig.* **2019**, *28*, S95–S104. [CrossRef]
31. Kibana: Explore, Visualize, Discover Data. Elastic. Available online: https://www.elastic.co/kibana (accessed on 22 August 2025).
32. Lucene Query Syntax | Kibana Guide [8.12] | Elastic. www.elastic.co. Available online: https://www.elastic.co/guide/en/kibana/current/lucene-query.html (accessed on 22 August 2025).
33. CrowdStrike Intelligence Team. Technical Analysis of the WhisperGate Malicious Bootloader | CrowdStrike. Crowdstrike.Com. 2024. Available online: https://www.crowdstrike.com/en-us/blog/technical-analysis-of-whispergate-malware/ (accessed on 22 August 2025).

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.